

✓ SARSA Algorithm

```
import numpy as np
import matplotlib.pyplot as plt
```

```
grid_size = 5

start = (0, 0)
goal = (4, 4)

actions = ['up', 'down', 'left', 'right']
```

```
Q = {}

for row in range(grid_size):
    for col in range(grid_size):
        Q[(row, col)] = {}

        for action in actions:
            Q[(row, col)][action] = 0
```

```
alpha = 0.1      # Learning rate
gamma = 0.9      # Discount factor
epsilon = 0.2    # Exploration rate

episodes = 500
```

```
def choose_action(state):

    if np.random.uniform(0, 1) < epsilon:
        return np.random.choice(actions)

    else:
        return max(Q[state], key=Q[state].get)
```

```
def next_state(state, action):

    row, col = state

    if action == 'up':
        row = max(row - 1, 0)

    elif action == 'down':
        row = min(row + 1, grid_size - 1)

    elif action == 'left':
        col = max(col - 1, 0)

    elif action == 'right':
        col = min(col + 1, grid_size - 1)

    return (row, col)
```

```
for episode in range(episodes):

    state = start
    action = choose_action(state)

    while state != goal:
```

```

    new_state = next_state(state, action)

    reward = 100 if new_state == goal else -1

    new_action = choose_action(new_state)

    # SARSA Update Rule
    Q[state][action] = Q[state][action] + alpha * (
        reward +
        gamma * Q[new_state][new_action] -
        Q[state][action]
    )

    state = new_state
    action = new_action

```

```

state = start

path = [state]

while state != goal:

    action = max(Q[state], key=Q[state].get)

    state = next_state(state, action)

    path.append(state)

print("Optimal Path:")
print(path)

```

```

Optimal Path:
[(0, 0), (0, 1), (1, 1), (2, 1), (2, 2), (2, 3), (2, 4), (3, 4), (4, 4)]

```

```

grid = np.zeros((grid_size, grid_size))

for position in path:
    grid[position] = 1

grid[start] = 2
grid[goal] = 3

plt.figure(figsize=(6,6))

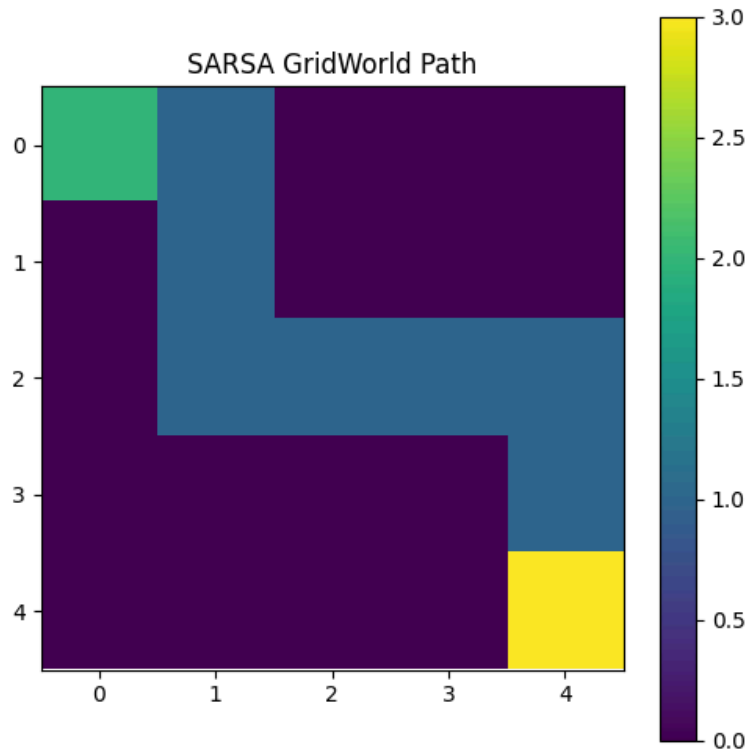
plt.imshow(grid, cmap='viridis')

plt.title("SARSA GridWorld Path")

plt.colorbar()

plt.show()

```



Start coding or [generate](#) with AI.